UNSW-NB15: Centralized vs Local vs Federated IDS under Temporal Shift and Client Heterogeneity

Publication-grade experiment notebook (v4)

Locked research question

> How does IDS generalization change when centralized, local and federated learning are evaluated under temporal distribution shift and increasing client-level data heterogeneity?

What this notebook runs (press *Runtime -> Run all*; it is resumable, see below)

Condition	Centralized	Local	FedAvg
Random stratified split (60/20/20) - baseline	yes		
Temporal (60/20/20 chronological)	yes		
Temporal + Dirichlet alpha = 10 / 1 / 0.5 / 0.1	yes (pooled, budget-matched)	yes	yes
Duplicate sensitivity (duplicates kept vs removed)	yes		yes
Auxiliary: unseen attack-family generalization	yes	yes	yes
Temporal split-fraction sensitivity (50/25/25, 70/10/20)	yes		

Every condition is repeated over **5 seeds** (42, 123, 456, 789, 101112).

Reported metrics. The detector is *binary* (benign vs attack), so the headline metrics are binary: Macro-F1, attack precision, attack recall, benign specificity, FPR and PR-AUC. Per attack family we report **detection recall only** (support, share of that family's flows flagged as attack). Per-family precision/F1 are *not* defined for a binary detector (the model never predicts a family) and are deliberately not reported. Aggregation: mean +/- SD over seeds, 95% t-intervals, paired mean differences with 95% CI and Hedges' g as the primary evidence, plus paired t-test and Wilcoxon signed-rank p-values with Holm correction as secondary evidence.

Design decisions (explicit, for the Methods section)

- Same MLP *architecture* (64-32) in every method. Claims about the **training paradigm** are restricted to the three *budget-matched* methods: `centralized_matched`, `local` and `fedavg` (all: `FED_ROUNDS x LOCAL_EPOCHS` epochs of `partial_fit`, no early stopping). `centralized_es` (early stopping, internal validation split) is kept only as a clearly-labelled **reference**, not as part of the paradigm comparison.
- **Local** = independent training: each client starts from a fresh random initialization and trains `FED_ROUNDS x LOCAL_EPOCHS` local epochs on its own

rows. **FedAvg** = the same primitive, `LOCAL_EPOCHS` epochs per round for `FED_ROUNDS` rounds, started from the shared global model. The only difference is independent vs collaborative training. (Inherent difference: FedAvg clients restart Adam each round, local/centralized models keep Adam state across epochs.)

- Local results are reported two ways: **`local_macro`** (every client equal weight = average node performance) and **`local_weighted`** (clients weighted by their number of training rows = traffic-level performance). All local models are scored on the same global test set, so weighting is by *training* rows.
- **Random stratified baseline**, not "IID": a stratified random split of network flows is not guaranteed to be IID (related flows, campaigns and sessions can sit on both sides).
- **Dirichlet partition** is a single, pure Dirichlet(alpha) draw per (seed, alpha) - *no* rejection on client size. Empty/tiny clients are possible at small alpha; they are reported (`client_heterogeneity.csv`) and clients without enough rows of both classes are not trained locally. FedAvg still includes every non-empty client.
- **Training sample protocol.** The temporal protocol is a *chronological holdout with stratified subsampling of the historical training block*: the first 60% of flows (by time) form the training block, from which a seeded sample (stratified by binary label, at most `MAX_TRAIN_ROWS`) is drawn. Nothing after the training boundary is ever sampled. Set `MAX_TRAIN_ROWS = None` to use the whole block.
- *Option A* preprocessing: "federated model training with shared feature preprocessing" - one preprocessor fitted on the (capped) training sample only; it never sees validation/test rows. Preprocessing is **not** decentralized, so this is *not* a privacy-preserving federated pipeline.
- The temporal split is *not* tuned for results; its fractions are justified against the capture structure (see `temporal_protocol_report.csv`) and checked by a sensitivity experiment.
- Hyper-parameters are fixed a priori. The validation partition is **reserved and untouched**: it is reported for the protocol but used for no model selection and no tuning; it is kept for possible future model selection.
- Duplicates: exact duplicate records can make evaluation optimistic when identical observations cross the train/test boundary. They are *not* described as "leakage" per se (repeated legitimate flows exist); the ablation quantifies the effect.
- Nothing here is claimed to be novel; that requires the literature check. "Unseen-family" results are an *auxiliary* experiment and *not* zero-day detection claims. Results hold for the evaluated MLP architecture only.

Resumable: each (seed, condition) unit is saved to `results_publication/runs/` (file names carry the notebook version, so results of older notebook versions are never reused). If Colab disconnects, run all again and finished units are skipped.

```
# ========================
# 0. IMPORTS
# ========================
import json, os, re, time, warnings, math, random, hashlib, platform, sys, d
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn, scipy
from scipy import stats

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             confusion_matrix, average_precision_score)
from sklearn.model_selection import train_test_split

try:
    from IPython.display import display
except Exception:
    display = print

warnings.filterwarnings("ignore")
print("scikit-learn", sklearn.__version__, "| pandas", pd.__version__, "| nu
```

```
scikit-learn 1.6.1 | pandas 2.2.3 | numpy 2.1.3 | scipy 1.16.3
```

```
# =========================
# 1. GOOGLE DRIVE
# =========================
try:
    from google.colab import drive
    drive.mount("/content/drive")
    PROJECT = Path("/content/drive/MyDrive/UNSW_Project")
except ImportError:
    PROJECT = Path(os.environ.get("UNSW_PROJECT", "./UNSW_Project"))

RAW = PROJECT / "data" / "raw"
RESULTS = PROJECT / "results_publication"
RUNS = RESULTS / "runs"
FIGURES = RESULTS / "figures"
TABLES = RESULTS / "tables"
for p in [RAW, RESULTS, RUNS, FIGURES, TABLES]:
    p.mkdir(parents=True, exist_ok=True)
print("Project:", PROJECT)
```

```
Mounted at /content/drive
Project: /content/drive/MyDrive/UNSW_Project
```

```
# =========================
# 2. CONFIGURATION  (everything that defines the experiment lives here)
# =========================
FILES = [RAW / f"UNSW-NB15_{i}.csv" for i in range(1, 5)]

COLS = [
    "srcip","sport","dstip","dsport","proto","state","dur","sbytes","dbytes"
    "sttl","dttl","sloss","dloss","service","Sload","Dload","Spkts","Dpkts",
    "dwin","stcpb","dtcpb","smeansz","dmeansz","trans_depth","res_bdy_len","
    "Djit","Stime","Ltime","Sintpkt","Dintpkt","tcprtt","synack","ackdat",
    "is_sm_ips_ports","ct_state_ttl","ct_flw_http_mthd","is_ftp_login","ct_f
    "ct_srv_src","ct_srv_dst","ct_dst_ltm","ct_src_ltm","ct_src_dport_ltm",
    "ct_dst_sport_ltm","ct_dst_src_ltm","attack_cat","Label"
]
CATEGORICAL = ["proto", "state", "service"]
IDENTIFIERS = ["srcip","dstip","sport","dsport"]
TIME_COLS = ["Stime", "Ltime"]
TARGET = "Label"
NON_FEATURES = set(IDENTIFIERS + TIME_COLS + ["attack_cat","Label","ts","fam
                                              "source_file","session","is_du
CLASSES = np.array([0, 1])

NOTEBOOK_VERSION = "v4-publication"
SESSION_GAP_SECONDS = 3600              # a gap larger than this starts a ne

# ---- run profile ------------------------------------------------------
# "full"  : the publication protocol (5 seeds x 4 alphas ...). Takes a while
# "smoke" : tiny caps / 2 seeds, only to check that the pipeline runs end-to
RUN_PROFILE = os.environ.get("UNSW_PROFILE", "full")
PROFILES = {
    "full":  dict(SEEDS=[42, 123, 456, 789, 101112], ALPHAS=[10, 1, 0.5, 0.1
                  MAX_TRAIN_ROWS=300_000, FED_ROUNDS=10, LOCAL_EPOCHS=1, LOC
```

```
                    MIN_CLIENT_ROWS=200, MIN_CLASS_ROWS_LOCAL=50, UNSEEN_MIN_R
    "smoke": dict(SEEDS=[42, 123], ALPHAS=[1, 0.1],
                  MAX_TRAIN_ROWS=6_000, FED_ROUNDS=3, LOCAL_EPOCHS=1, LOCAL_
                  MIN_CLIENT_ROWS=50, MIN_CLASS_ROWS_LOCAL=20, UNSEEN_MIN_RO
}
assert RUN_PROFILE in PROFILES, f"unknown profile {RUN_PROFILE}"
globals().update(PROFILES[RUN_PROFILE])

N_CLIENTS = 5
PARTITION_BY = "family"                 # Dirichlet label-skew over the 10 t
PARTITION_MIN_ROWS = None               # None = PURE Dirichlet(alpha), one
                                        # An int would redraw until every cl
# MIN_CLIENT_ROWS (profile) is now only a REPORTING threshold: clients below
SPLIT_FRACTIONS = (0.60, 0.20, 0.20)    # train / validation / test, used fo
                                        # validation is RESERVED and untouch
SPLIT_SENSITIVITY = {"50_25_25": (0.50, 0.25, 0.25), "70_10_20": (0.70, 0.10
HIDDEN = (64, 32)

RUN_DUP_ABLATION = True
RUN_SPLIT_SENSITIVITY = True
RUN_UNSEEN_FAMILY = True                # auxiliary experiment; switch off t
ALPHA_FOR_ABLATIONS = 0.5               # heterogeneity level used for dupli
N_UNSEEN = 2
TRACK_CURVE_SEEDS = [SEEDS[0]]          # per-round FedAvg curves only for t
MATCHED_EPOCHS = FED_ROUNDS * LOCAL_EPOCHS

print("Profile:", RUN_PROFILE, "| seeds:", SEEDS, "| alphas:", ALPHAS)
```

```
Profile: full | seeds: [42, 123, 456, 789, 101112] | alphas: [10, 1, 0.5, 0.1
```

```
# =========================
# 3. VERIFICATION HELPERS
# =========================
AUDIT = {}
FAILURES = []

def check(condition, message, fatal=True, quiet=False):
    if condition:
        if not quiet:
            print("[PASS]", message)
        return True
    print("[FAIL]", message)
    FAILURES.append(message)
    if fatal:
        raise RuntimeError("Verification failed: " + message)
    return False

def warn(message):
    print("[WARN]", message)

def record(key, value):
    AUDIT[key] = value
    return value
```

```
def json_default(o):
    if hasattr(o, "item"):
        return o.item()
    if isinstance(o, (set, tuple)):
        return list(o)
    return str(o)

check(tuple(int(x) for x in sklearn.__version__.split(".")[:2]) >= (1, 2),
      f"scikit-learn {sklearn.__version__} supports OneHotEncoder(sparse_out
```

```
[PASS] scikit-learn 1.6.1 supports OneHotEncoder(sparse_output=...).
True
```

```
# =========================
# 4. RAW FILES: EXISTENCE, WIDTH, PROVENANCE (SHA-256)
# =========================
missing = [str(f) for f in FILES if not f.exists()]
check(len(missing) == 0, "All four UNSW-NB15 raw CSV files exist." if not mi

def sha256_of(path, chunk=1 << 22):
    h = hashlib.sha256()
    with open(path, "rb") as fh:
        while True:
            b = fh.read(chunk)
            if not b:
                break
            h.update(b)
    return h.hexdigest()

SCHEMA_OK = True
FILE_PROVENANCE = []
for f in FILES:
    head = pd.read_csv(f, header=None, nrows=5, dtype=str, encoding="latin-1
    width_ok = head.shape[1] == len(COLS)
    no_header = str(head.iloc[0, 0]).strip().lower() != "srcip"
    SCHEMA_OK &= bool(width_ok and no_header)
    check(width_ok, f"{f.name}: {head.shape[1]} columns (expected {len(COLS)
    check(no_header, f"{f.name}: no header row.")
    FILE_PROVENANCE.append({"file": f.name, "bytes": f.stat().st_size, "sha2
    print("   ", f.name, f"{f.stat().st_size/1024**2:.1f} MB", FILE_PROVENAN
```

```
[PASS] All four UNSW-NB15 raw CSV files exist.
[PASS] UNSW-NB15_1.csv: 49 columns (expected 49).
[PASS] UNSW-NB15_1.csv: no header row.
    UNSW-NB15_1.csv 161.2 MB 7d851bbeabd27894 ...
[PASS] UNSW-NB15_2.csv: 49 columns (expected 49).
[PASS] UNSW-NB15_2.csv: no header row.
    UNSW-NB15_2.csv 157.6 MB 6130ad02873cc606 ...
[PASS] UNSW-NB15_3.csv: 49 columns (expected 49).
[PASS] UNSW-NB15_3.csv: no header row.
    UNSW-NB15_3.csv 147.4 MB ae990a96c3dfcd42 ...
[PASS] UNSW-NB15_4.csv: 49 columns (expected 49).
[PASS] UNSW-NB15_4.csv: no header row.
    UNSW-NB15_4.csv 93.1 MB cdf563692d51d405 ...
```

```
# =========================
# 5. LOAD (memory-safe: identifiers are never loaded)
# =========================
STR_COLS = ["proto","state","service","attack_cat","ct_flw_http_mthd","is_ft
LOAD_COLS = [c for c in COLS if c not in IDENTIFIERS]

frames, row_counts = [], {}
for f in FILES:
    d = pd.read_csv(f, header=None, names=COLS, usecols=LOAD_COLS,
                    dtype={c: str for c in STR_COLS}, encoding="latin-1", lo
    d["source_file"] = f.name
    frames.append(d)
    row_counts[f.name] = len(d)
    print(f"{f.name}: {len(d):,} rows")

raw = pd.concat(frames, ignore_index=True)
del frames
SCHEMA_OK = bool(SCHEMA_OK and set(raw.columns) - {"source_file"} == set(LOA
check(SCHEMA_OK, "Expected UNSW-NB15 schema present.")
ROWS_LOADED = int(len(raw))
record("rows_loaded", ROWS_LOADED); record("row_counts", row_counts)
print("Total loaded:", f"{ROWS_LOADED:,}")
```

```
UNSW-NB15_1.csv: 700,001 rows
UNSW-NB15_2.csv: 700,001 rows
UNSW-NB15_3.csv: 700,001 rows
UNSW-NB15_4.csv: 440,044 rows
[PASS] Expected UNSW-NB15 schema present.
Total loaded: 2,540,047
```

```
# =========================
# 6. CLEAN + NORMALIZE
# =========================
df = raw
del raw

TEXT_COLS = set(CATEGORICAL + ["attack_cat", "source_file"])
numeric_cols = [c for c in LOAD_COLS if c not in TEXT_COLS]
for c in numeric_cols:
    df[c] = pd.to_numeric(df[c], errors="coerce")
for c in ["ct_flw_http_mthd", "is_ftp_login", "ct_ftp_cmd"]:     # legitimat
    df[c] = df[c].fillna(0)
for c in CATEGORICAL:
    df[c] = df[c].astype(str).str.strip().str.lower()

fam = df["attack_cat"].fillna("").astype(str).str.strip()
fam = fam.str.replace(r"\s+", " ", regex=True)
fam = fam.replace({"": "Benign", "Backdoors": "Backdoor"})
df["family"] = fam

bad_numeric = df[numeric_cols].isna().any(axis=1)
N_INVALID = int(bad_numeric.sum())
record("invalid_numeric_rows", N_INVALID)
if N_INVALID:
    warn(f"{N_INVALID:,} rows with non-numeric values dropped ({N_INVALID/le
```

```
check(N_INVALID / max(len(df), 1) < 0.01, "Fewer than 1% invalid rows (else

df = df[~bad_numeric].copy()
df[TARGET] = df[TARGET].astype(int)
check(set(df[TARGET].unique()).issubset({0, 1}), "Labels are binary 0/1.")
df["ts"] = pd.to_datetime(df["Stime"], unit="s", errors="coerce")
check(df["ts"].notna().all(), "All timestamps parsed.")
df = df.sort_values("ts", kind="mergesort").reset_index(drop=True)
ROWS_AFTER_CLEAN = int(len(df))
print("Rows after cleaning:", f"{ROWS_AFTER_CLEAN:,}")
```

```
[PASS] Fewer than 1% invalid rows (else loader/schema is wrong).
[PASS] Labels are binary 0/1.
[PASS] All timestamps parsed.
Rows after cleaning: 2,540,047
```

```
# ========================
# 7. DUPLICATE AUDIT (marked, not yet discarded: needed for the duplicate ab
# ========================
feature_cols_for_duplicate = [c for c in df.columns if c not in NON_FEATURES
dup_mask = df.duplicated(subset=feature_cols_for_duplicate + ["Label", "fami
N_DUP = int(dup_mask.sum())
record("duplicates_detected", N_DUP)
record("duplicate_rate", N_DUP / max(ROWS_AFTER_CLEAN, 1))
print(f"Duplicate rows (identical features+label+family, earliest kept): {N_

df_full = df                                    # chronological, duplicates r
df_full["is_dup"] = dup_mask.to_numpy()
df = df_full[~df_full["is_dup"]].reset_index(drop=True)   # PRIMARY analysis
del dup_mask
ROWS_FINAL = int(len(df))
record("rows_after_deduplication", ROWS_FINAL)
print("Primary analysis set:", f"{ROWS_FINAL:,}", "| with-duplicates set:",

# label/family consistency
bad_0 = int(((df.Label == 0) & (df.family != "Benign")).sum())
bad_1 = int(((df.Label == 1) & (df.family == "Benign")).sum())
check(bad_0 == 0 and bad_1 == 0, f"Label/family consistency ({bad_0}+{bad_1}
print(df["family"].value_counts())
```

```
Duplicate rows (identical features+label+family, earliest kept): 513,438 (20.
Primary analysis set: 2,026,609 | with-duplicates set: 2,540,047
[PASS] Label/family consistency (0+0 contradictions).
family
Benign            1938355
Exploits            27434
Fuzzers             20960
Generic             18830
Reconnaissance       9991
DoS                  5500
Analysis             2032
Backdoor             1880
Shellcode            1456
Worms                 171
Name: count, dtype: int64
```

```
# ========================
# 8. CAPTURE-SESSION AUDIT
# ========================
gaps = df["ts"].diff().dt.total_seconds()
df["session"] = (gaps > SESSION_GAP_SECONDS).cumsum().astype(int)
sessions = (df.groupby("session").agg(start=("ts","min"), end=("ts","max"),
sessions["duration_hours"] = (sessions["end"] - sessions["start"]).dt.total_
record("num_candidate_sessions", int(len(sessions)))
record("largest_gap_hours", float(gaps.max() / 3600) if len(gaps) > 1 else 0
print(sessions.to_string(index=False))
sessions.to_csv(TABLES / "capture_sessions.csv", index=False)
coverage = pd.crosstab(df["session"], df["family"])
coverage.to_csv(TABLES / "session_family_counts.csv")
print(coverage.to_string())
check(df["ts"].is_monotonic_increasing, "Chronologically sorted.")
```

```
 session               start                 end    rows  duration_hours
       0 2015-01-22 11:49:37 2015-01-23 00:25:24 1015988       12.596389
       1 2015-02-18 00:23:28 2015-02-18 12:21:08 1010621       11.961111
family   Analysis  Backdoor   Benign   DoS  Exploits  Fuzzers  Generic  Recon
session
0             299       297  1003080   823      4040     3911     1896
1            1733      1583   935275  4677     23394    17049    16934
[PASS] Chronologically sorted.
True
```

```
# ========================
# 9. FEATURES, PREPROCESSOR, METRICS
# ========================
MODEL_FEATURES = [c for c in df.columns if c not in NON_FEATURES]
numeric_features = [c for c in MODEL_FEATURES if c not in CATEGORICAL]
categorical_features = [c for c in MODEL_FEATURES if c in CATEGORICAL]

LEAKAGE_STATIC = {
    "no_ip_or_port_identifiers": not (set(MODEL_FEATURES) & set(IDENTIFIERS)
    "no_timestamps_as_features": not (set(MODEL_FEATURES) & (set(TIME_COLS)
    "no_target_derived_features": not (set(MODEL_FEATURES) & {"Label", "atta
    "no_session_or_bookkeeping_features": not (set(MODEL_FEATURES) & {"sessi
}
for k, v in LEAKAGE_STATIC.items():
    check(v, f"Leakage audit: {k}.")
check(len(MODEL_FEATURES) > 10, f"{len(MODEL_FEATURES)} model features.")
check(all(pd.api.types.is_numeric_dtype(df[c]) for c in numeric_features), "
check(int(df[MODEL_FEATURES].isna().sum().sum()) == 0, "No missing values in
check(np.isfinite(df[numeric_features].to_numpy(dtype=float)).all(), "No inf

def make_preprocessor():
    num = Pipeline([("imputer", SimpleImputer(strategy="median")), ("scaler"
    cat = Pipeline([("imputer", SimpleImputer(strategy="most_frequent")),
                    ("onehot", OneHotEncoder(handle_unknown="ignore", sparse
    return ColumnTransformer([("num", num, numeric_features), ("cat", cat, c

FAM_NAMES = sorted(df_full["family"].unique().tolist())
FAM_TO_CODE = {f: i for i, f in enumerate(FAM_NAMES)}
```

```
BENIGN_CODE = FAM_TO_CODE["Benign"]

def metrics_from(pred, prob, y):
    tn, fp, fn, tp = confusion_matrix(y, pred, labels=[0, 1]).ravel()
    try:
        pr = average_precision_score(y, prob)
    except Exception:
        pr = np.nan
    return {
        "accuracy": accuracy_score(y, pred),
        "precision": precision_score(y, pred, zero_division=0),
        "recall": recall_score(y, pred, zero_division=0),
        "specificity": tn / max(tn + fp, 1),
        "macro_f1": f1_score(y, pred, average="macro", labels=[0, 1], zero_d
        "fpr": fp / max(fp + tn, 1),
        "pr_auc": pr,
        "tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp),
    }

def family_table(pred, fam_codes):
    """Per-family DETECTION metrics for a BINARY (benign vs attack) detector
       attack family f : support and recall = (flows of f flagged as attack)
       Benign          : support and specificity = (benign flows kept benign
       Per-family precision / F1 are intentionally NOT computed: the model n
       so flows of other attack families flagged as 'attack' are not false p
       classifier and any 'family precision' built from this output would be
    K = len(FAM_NAMES)
    n = np.bincount(fam_codes, minlength=K)
    flagged = np.bincount(fam_codes[pred == 1], minlength=K)
    rows = []
    for k, name in enumerate(FAM_NAMES):
        if n[k] == 0:
            continue
        if k == BENIGN_CODE:
            rows.append(dict(family=name, support=int(n[k]), recall=np.nan,
                             specificity=float((n[k] - flagged[k]) / n[k])))
        else:
            rows.append(dict(family=name, support=int(n[k]), recall=float(fl
                             specificity=np.nan))
    return rows

def evaluate(model, X, y, fam_codes):
    pred = model.predict(X)
    prob = model.predict_proba(X)[:, 1]
    return metrics_from(pred, prob, y), family_table(pred, fam_codes)
print("Features:", len(MODEL_FEATURES), "| families:", FAM_NAMES)
```

```
[PASS] Leakage audit: no_ip_or_port_identifiers.
[PASS] Leakage audit: no_timestamps_as_features.
[PASS] Leakage audit: no_target_derived_features.
[PASS] Leakage audit: no_session_or_bookkeeping_features.
[PASS] 41 model features.
[PASS] Numeric features are numeric.
[PASS] No missing values in features.
```

```
[PASS] No infinite values in features.
Features: 41 | families: ['Analysis', 'Backdoor', 'Benign', 'DoS', 'Exploits'
```

```
# ========================
# 10. EVALUATION CONTEXTS AND SPLIT PROTOCOLS
# ========================
FEATURE_HASH_COLS = MODEL_FEATURES + [TARGET, "family"]

def build_context(frame, name):
    frame = frame.reset_index(drop=True)
    return dict(
        name=name, n=len(frame),
        X=frame[MODEL_FEATURES].copy(),
        y=frame[TARGET].to_numpy(dtype=int),
        fam=frame["family"].map(FAM_TO_CODE).to_numpy(dtype=int),
        ts=frame["ts"].to_numpy(),
        session=frame["session"].to_numpy() if "session" in frame else None,
        h=pd.util.hash_pandas_object(frame[FEATURE_HASH_COLS], index=False).
    )

def temporal_split(n, fr):
    a, b = int(n * fr[0]), int(n * (fr[0] + fr[1]))
    return np.arange(0, a), np.arange(a, b), np.arange(b, n)

def random_split(y, fr, seed):
    idx = np.arange(len(y))
    tr, tmp = train_test_split(idx, train_size=fr[0], stratify=y, random_sta
    va, te = train_test_split(tmp, test_size=fr[2] / (fr[1] + fr[2]), strati
    return np.sort(tr), np.sort(va), np.sort(te)

def overlap(ctx, a, b):
    """Rows of b that have an identical (features+label+family) row in a."""
    return int(np.isin(ctx["h"][b], ctx["h"][a]).sum())

def audit_split(ctx, tr, va, te, label):
    out = dict(split=label, context=ctx["name"],
               train_test_dup=overlap(ctx, tr, te), val_test_dup=overlap(ctx
               train_val_dup=overlap(ctx, tr, va),
               index_overlap=int(len(np.intersect1d(tr, te)) + len(np.inters
               test_rows=int(len(te)))
    out["test_dup_rate"] = out["train_test_dup"] / max(len(te), 1)
    return out

CTX = build_context(df, "dedup")
TEMPORAL_SPLIT = temporal_split(CTX["n"], SPLIT_FRACTIONS)
tr_t, va_t, te_t = TEMPORAL_SPLIT

check(CTX["ts"][tr_t].max() <= CTX["ts"][va_t].min() and CTX["ts"][va_t].max
      "Temporal split is chronologically ordered (train <= validation <= tes
STATIC_AUDITS = [audit_split(CTX, *TEMPORAL_SPLIT, "temporal")]
check(STATIC_AUDITS[0]["index_overlap"] == 0 and STATIC_AUDITS[0]["train_tes
      and STATIC_AUDITS[0]["val_test_dup"] == 0,
      "Dedup temporal split: no index overlap and no duplicate train/test or
for name, idxs in {"train": tr_t, "validation": va_t, "test": te_t}.items():
```

```
    c = np.bincount(CTX["y"][idxs], minlength=2)
    check(c.min() > 0, f"Temporal {name} split has both classes {c.tolist()}

# --- protocol report: how the chronological fractions map onto the capture
proto_rows = []
for nm, idxs in {"train": tr_t, "validation": va_t, "test": te_t}.items():
    sess_share = pd.Series(CTX["session"][idxs]).value_counts(normalize=True
    proto_rows.append(dict(split=nm, rows=len(idxs), start=str(pd.Timestamp(
                           end=str(pd.Timestamp(CTX["ts"][idxs].max())),
                           attack_rate=float(CTX["y"][idxs].mean()),
                           session_share="; ".join(f"s{k}={v:.1%}" for k, v
proto_report = pd.DataFrame(proto_rows)
proto_report.to_csv(TABLES / "temporal_protocol_report.csv", index=False)
print(proto_report.to_string(index=False))

fam_by_split = pd.DataFrame({nm: pd.Series(CTX["fam"][i]).map(dict(enumerate
                             for nm, i in {"train": tr_t, "validation": va_t
fam_by_split.to_csv(TABLES / "temporal_family_by_split.csv")
print(fam_by_split)

# --- data lineage funnel
lineage = pd.DataFrame([
    ("raw rows loaded", ROWS_LOADED, "four UNSW-NB15 CSV parts, concatenated
    ("invalid rows removed", -N_INVALID, "non-numeric values in numeric colu
    ("rows after cleaning", ROWS_AFTER_CLEAN, "sorted chronologically by flo
    ("duplicate rows removed", -N_DUP, "identical features + label + family;
    ("final analysis rows", ROWS_FINAL, "primary dataset (duplicates removed
    ("capture sessions", len(sessions), f"gap > {SESSION_GAP_SECONDS/3600:g}
    ("temporal train rows (historical block)", len(tr_t), f"first {SPLIT_FRA
    ("temporal train rows actually used", min(len(tr_t), MAX_TRAIN_ROWS or l
     "seeded sample of the historical block, stratified by binary label (cap
    ("temporal validation rows", len(va_t), f"next {SPLIT_FRACTIONS[1]:.0%};
    ("temporal test rows", len(te_t), f"last {SPLIT_FRACTIONS[2]:.0%}; ident
], columns=["stage", "rows", "justification"])
lineage.to_csv(TABLES / "data_lineage.csv", index=False)
display(lineage)

# --- unseen-family candidate selection (rule fixed before any model is trai
cand = []
for k, name in enumerate(FAM_NAMES):
    if k == BENIGN_CODE:
        continue
    m = CTX["fam"] == k
    sh = np.bincount(CTX["session"][m]) if m.any() else np.array([0])
    cand.append(dict(family=name, train_rows=int((CTX["fam"][tr_t] == k).sum
                     test_rows=int((CTX["fam"][te_t] == k).sum()),
                     n_sessions=int((sh > 0).sum()), max_session_share=float
cand = pd.DataFrame(cand)
cand.to_csv(TABLES / "unseen_family_candidates.csv", index=False)
elig = cand[(cand.train_rows >= UNSEEN_MIN_ROWS) & (cand.test_rows >= UNSEEN
pref = elig[elig.max_session_share <= 0.9]
if len(pref) < N_UNSEEN:
    warn("Fewer than N_UNSEEN families are spread over sessions; using tempo
    pref = elig
UNSEEN_FAMILIES = pref.sort_values("train_rows").head(N_UNSEEN).family.tolis
```

```
record("unseen_families", UNSEEN_FAMILIES)
print("Unseen-family experiments:", UNSEEN_FAMILIES or "none eligible")
display(cand)
```

```
[PASS] Temporal split is chronologically ordered (train <= validation <= tes
[PASS] Dedup temporal split: no index overlap and no duplicate train/test or
[PASS] Temporal train split has both classes [1188591, 27374].
[PASS] Temporal validation split has both classes [373084, 32238].
[PASS] Temporal test split has both classes [376680, 28642].
     split    rows               start                 end  attack_rate
     train 1215965 2015-01-22 11:49:37 2015-02-18 03:11:29     0.022512 s0=8
validation  405322 2015-02-18 03:11:29 2015-02-18 07:46:45     0.079537
      test  405322 2015-02-18 07:46:45 2015-02-18 12:21:08     0.070665
                train  validation    test
Analysis            596         822     614
Backdoor            613         646     621
Benign          1188591      373084  376680
DoS                1732        2033    1735
Exploits           8683        9684    9067
Fuzzers            7214        7134    6612
Generic            4869        7685    6276
Reconnaissance     3134        3656    3201
Shellcode           479         518     459
Worms                54          60      57
```

	stage	rows	justification
0	raw rows loaded	2540047	four UNSW-NB15 CSV parts, concatenated
1	invalid rows removed	0	non-numeric values in numeric columns (must st...
2	rows after cleaning	2540047	sorted chronologically by flow start time
3	duplicate rows removed	-513438	identical features + label + family; earliest ...
4	final analysis rows	2026609	primary dataset (duplicates removed)
5	capture sessions	2	gap > 1 hour starts a new session
6	temporal train rows (historical block)	1215965	first 60% of rows in time
7	temporal train rows actually used	300000	seeded sample of the historical block, stratif...
8	temporal validation rows	405322	next 20%; reserved and untouched (no model sel...
9	temporal test rows	405322	last 20%; identical for every method

```
Unseen-family experiments: ['DoS', 'Reconnaissance']
```

	family	train_rows	test_rows	n_sessions	max_session_share
0	Analysis	596	614	2	0.852854
1	Backdoor	613	621	2	0.842021
2	DoS	1732	1735	2	0.850364
3	Exploits	8683	9067	2	0.852737
4	Fuzzers	7214	6612	2	0.813406
5	Generic	4869	6276	2	0.899310

Next steps: Generate code with lineage | New interactive sheet | Generate code with cand

```
# ========================
# 11. EXPERIMENT ENGINE
# ========================
def cap_indices(idx, y, max_rows, seed):
    """Seeded sample of the (historical) training block, stratified by binar
       max_rows=None -> the whole block is used. Never draws rows outside `i
    idx = np.asarray(idx)
    if max_rows is None or len(idx) <= max_rows:
        return idx.copy()
    keep, _ = train_test_split(idx, train_size=max_rows, stratify=y[idx], ra
    return np.sort(keep)

def prepare(ctx, tr, te, seed):
    """Seeded stratified cap of the training rows, ONE shared preprocessor f
    cap = cap_indices(tr, ctx["y"], MAX_TRAIN_ROWS, seed)
    leak_free = bool(np.intersect1d(cap, te).size == 0 and np.isin(cap, tr).
    check(leak_free, "Preprocessor fit rows are inside train and disjoint fr
    pre = make_preprocessor()
    Xtr = pre.fit_transform(ctx["X"].iloc[cap]).astype(np.float32)
    Xte = pre.transform(ctx["X"].iloc[te]).astype(np.float32)
    check(np.isfinite(Xtr).all() and np.isfinite(Xte).all(), "Transformed ma
    return dict(cap=cap, Xtr=Xtr, ytr=ctx["y"][cap], ftr=ctx["fam"][cap],
                Xte=Xte, yte=ctx["y"][te], fte=ctx["fam"][te],
                test_hash=hashlib.md5(np.asarray(te).tobytes()).hexdigest()[
                leak_free=leak_free, context=ctx["name"])

def make_mlp(seed, max_iter=1, early_stopping=False):
    return MLPClassifier(hidden_layer_sizes=HIDDEN, activation="relu", solve
                         batch_size=256, learning_rate_init=1e-3, max_iter=m
                         early_stopping=early_stopping, validation_fraction=

# ---------- centralized baselines (same architecture) ----------
def run_centralized_es(prep, seed):
    """REFERENCE ONLY: early stopping + internal validation split. Not budge
       the training-paradigm comparison (that uses centralized_matched / loc
    t = time.time()
    m = make_mlp(seed, max_iter=LOCAL_MAX_ITER, early_stopping=True)
    m.fit(prep["Xtr"], prep["ytr"])
    met, ft = evaluate(m, prep["Xte"], prep["yte"], prep["fte"])
    return met, ft, time.time() - t

def run_centralized_matched(prep, seed):
    """Pooled data, same sample FedAvg's clients hold, same number of epochs
    t = time.time()
    m = make_mlp(seed)
    for _ in range(MATCHED_EPOCHS):
        m.partial_fit(prep["Xtr"], prep["ytr"], classes=CLASSES)
    met, ft = evaluate(m, prep["Xte"], prep["yte"], prep["fte"])
    return met, ft, time.time() - t

# ---------- Dirichlet partition (pure draw; optional constrained variant) -
```

```
def dirichlet_partition(labels, n_clients, alpha, seed, min_size=None):
    """Dirichlet(alpha) label-skew partition: for every label (traffic famil
       ~ Dirichlet(alpha * 1) and split that label's rows accordingly.
       min_size=None (MAIN protocol): exactly ONE draw per (seed, alpha) - t
       min_size=int (constrained SENSITIVITY variant only): redraw until eve
       that samples Dirichlet conditioned on the constraint and must be desc
    rng = np.random.default_rng(seed)
    labels = np.asarray(labels)
    max_attempts = 1 if min_size is None else 200
    for attempt in range(1, max_attempts + 1):
        parts = [[] for _ in range(n_clients)]
        for cls in np.unique(labels):
            pos = np.where(labels == cls)[0]
            rng.shuffle(pos)
            p = rng.dirichlet(np.repeat(alpha, n_clients))
            cuts = (np.cumsum(p) * len(pos)).astype(int)[:-1]
            for k, chunk in enumerate(np.split(pos, cuts)):
                parts[k].extend(chunk.tolist())
        if min_size is None or min(len(x) for x in parts) >= min_size:
            return [np.array(sorted(x), dtype=int) for x in parts], attempt
    raise RuntimeError(f"No partition with >= {min_size} rows/client found f

def heterogeneity(parts, fam_codes, y):
    from scipy.spatial.distance import jensenshannon
    K = len(FAM_NAMES)
    sizes = [len(p) for p in parts]
    nonempty = [p for p in parts if len(p) > 0]
    dist = [np.bincount(fam_codes[p], minlength=K) / len(p) for p in nonempt
    pair = [jensenshannon(dist[i], dist[j], base=2) ** 2 for i in range(len(
    return dict(js_divergence_mean=float(np.mean(pair)) if pair else np.nan,
                attack_rate_sd=float(np.std([y[p].mean() for p in nonempty])
                min_rows=int(min(sizes)), max_rows=int(max(sizes)),
                empty_clients=int(sum(s == 0 for s in sizes)),
                clients_below_min_rows=int(sum(s < MIN_CLIENT_ROWS for s in
                client_rows="/".join(str(s) for s in sizes))

# ---------- FedAvg ----------
def get_weights(m):
    return [x.copy() for x in m.coefs_], [x.copy() for x in m.intercepts_]

def set_weights(m, w):
    m.coefs_ = [x.copy() for x in w[0]]
    m.intercepts_ = [x.copy() for x in w[1]]

def weighted_average(sets, sizes):
    tot = float(sum(sizes))
    coefs = [sum(s[0][l] * n for s, n in zip(sets, sizes)) / tot for l in ra
    ints = [sum(s[1][l] * n for s, n in zip(sets, sizes)) / tot for l in ran
    return coefs, ints

def weights_finite(w):
    return all(np.isfinite(a).all() for a in w[0] + w[1])

def local_from_global(w, seed, Xi, yi):
    m = make_mlp(seed)
```

```
        m.partial_fit(Xi, yi, classes=CLASSES)          # builds layers; weights
        set_weights(m, w)
        if hasattr(m, "_optimizer"):                    # fresh Adam state every
            del m._optimizer
        return m

    def run_fedavg(client_arrays, prep, seed, track):
        t0 = time.time()
        Xi, yi = prep["Xtr"][:2], prep["ytr"][:2]
        gm = make_mlp(seed)
        gm.partial_fit(Xi, yi, classes=CLASSES)
        gw = get_weights(gm)
        hist, finite = [], True
        for rnd in range(1, FED_ROUNDS + 1):
            sets, sizes = [], []
            for Xc, yc in client_arrays:
                if len(Xc) == 0:
                    continue
                lm = local_from_global(gw, seed, Xi, yi)
                for _ in range(LOCAL_EPOCHS):
                    lm.partial_fit(Xc, yc, classes=CLASSES)
                sets.append(get_weights(lm)); sizes.append(len(Xc))
            check(len(sets) > 0, f"FedAvg round {rnd}: a client completed.", qui
            gw = weighted_average(sets, sizes)
            finite &= bool(weights_finite(gw))
            check(finite, f"FedAvg round {rnd}: aggregated weights finite.", qui
            set_weights(gm, gw)
            if track and rnd < FED_ROUNDS:
                pred = gm.predict(prep["Xte"]); prob = gm.predict_proba(prep["Xt
                hist.append(dict(metrics_from(pred, prob, prep["yte"]), round=rn
        met, ft = evaluate(gm, prep["Xte"], prep["yte"], prep["fte"])
        hist.append(dict(met, round=FED_ROUNDS))
        return met, ft, hist, finite, time.time() - t0

    # ---------- local-only (independent training) ----------
    def run_local(client_arrays, prep, seed):
        """Independent training with the SAME primitive FedAvg clients use: fres
           MATCHED_EPOCHS (= FED_ROUNDS x LOCAL_EPOCHS) epochs of partial_fit on
           no early stopping, no internal validation split. Every local model is
        out, skipped = [], []
        for cid, (Xc, yc) in enumerate(client_arrays):
            counts = np.bincount(yc.astype(int), minlength=2) if len(yc) else np
            if counts.min() < MIN_CLASS_ROWS_LOCAL:
                skipped.append(dict(client=cid, benign=int(counts[0]), attack=in
                continue
            t = time.time()
            m = make_mlp(seed * 10 + cid)
            for _ in range(MATCHED_EPOCHS):
                m.partial_fit(Xc, yc, classes=CLASSES)
            met, ft = evaluate(m, prep["Xte"], prep["yte"], prep["fte"])
            out.append((cid, len(yc), met, ft, time.time() - t))
        return out, skipped

    # ---------- bookkeeping ----------
    def new_store():
```

```
    return dict(rows=[], family=[], curve=[], hetero=[], audit=[], meta=[])

def log_run(store, condition, method, seed, prep, met, ft, train_rows, secon
    check(np.isfinite(met["macro_f1"]), f"{condition}/{method}/seed{seed}: f
    base = dict(condition=condition, method=method, seed=int(seed), alpha=al
    store["rows"].append(dict(base, train_rows=int(train_rows), test_rows=in
                              test_hash=prep["test_hash"], seconds=float(sec
    for r in ft:
        store["family"].append(dict(base, train_rows=int(train_rows), **r))

PREP = {}
def get_prep(seed, proto):
    key = (seed, proto)
    if key not in PREP:
        PREP.clear()
        PREP[key] = prepare(CTX, TEMPORAL_SPLIT[0], TEMPORAL_SPLIT[2], seed)
    return PREP[key]

def fl_experiment(store, condition, prep, seed, alpha, track, train_rows_not
    labels = prep["ftr"] if PARTITION_BY == "family" else prep["ytr"]
    parts, attempts = dirichlet_partition(labels, N_CLIENTS, alpha, seed, PA
    allp = np.concatenate(parts)
    check(len(allp) == len(prep["ytr"]) and len(np.unique(allp)) == len(allp
          "Every training row belongs to exactly one client.", quiet=True)
    client_arrays = [(prep["Xtr"][p], prep["ytr"][p]) for p in parts]
    het = heterogeneity(parts, prep["ftr"], prep["ytr"])
    met, ft, hist, finite, secs = run_fedavg(client_arrays, prep, seed, trac
    log_run(store, condition, "fedavg", seed, prep, met, ft, len(allp), secs
    store["meta"].append(dict(condition=condition, seed=int(seed), alpha=alp
    if track:
        for h in hist:
            store["curve"].append(dict(condition=condition, seed=int(seed),
    loc, skipped = run_local(client_arrays, prep, seed)
    for cid, n, m_, f_, s_ in loc:
        log_run(store, condition, "local", seed, prep, m_, f_, n, s_, alpha=
    store["hetero"].append(dict(condition=condition, seed=int(seed), alpha=a
                                local_trained=len(loc), local_skipped=len(sk
    return met

# ---------- units of work ----------
def unit_core(seed):
    store = new_store()
    for proto in ("random_stratified", "temporal"):
        if proto == "random_stratified":
            tr, va, te = random_split(CTX["y"], SPLIT_FRACTIONS, seed)
            prep = prepare(CTX, tr, te, seed)
        else:
            tr, va, te = TEMPORAL_SPLIT
            prep = get_prep(seed, "temporal")
        a = audit_split(CTX, tr, va, te, proto); a["seed"] = int(seed)
        store["audit"].append(a)
        check(a["index_overlap"] == 0 and a["train_test_dup"] == 0 and a["va
              f"seed {seed} {proto}: no index or duplicate overlap between s
        for method, fn in (("centralized_es", run_centralized_es), ("central
            met, ft, secs = fn(prep, seed)
```

```
                log_run(store, proto, method, seed, prep, met, ft, len(prep["ytr
            if proto == "random_stratified":
                del prep
        return store

    def unit_alpha(seed, alpha, track):
        store = new_store()
        prep = get_prep(seed, "temporal")
        fl_experiment(store, f"temporal_a{alpha:g}", prep, seed, alpha, track)
        return store

    CTX_DUP = {}
    def unit_dups(seed):
        store = new_store()
        if "ctx" not in CTX_DUP:
            CTX_DUP["ctx"] = build_context(df_full, "with_duplicates")
            CTX_DUP["split"] = temporal_split(CTX_DUP["ctx"]["n"], SPLIT_FRACTIO
        ctx, (tr, va, te) = CTX_DUP["ctx"], CTX_DUP["split"]
        a = audit_split(ctx, tr, va, te, "temporal_with_duplicates"); a["seed"]
        store["audit"].append(a)                                    # overlap is
        prep = prepare(ctx, tr, te, seed)
        met, ft, secs = run_centralized_matched(prep, seed)
        log_run(store, "temporal_with_duplicates", "centralized_matched", seed,
        labels = prep["ftr"] if PARTITION_BY == "family" else prep["ytr"]
        parts, attempts = dirichlet_partition(labels, N_CLIENTS, ALPHA_FOR_ABLAT
        arrays = [(prep["Xtr"][p], prep["ytr"][p]) for p in parts]
        met, ft, hist, finite, secs = run_fedavg(arrays, prep, seed, False)
        log_run(store, "temporal_with_duplicates", "fedavg", seed, prep, met, ft
        store["meta"].append(dict(condition="temporal_with_duplicates", seed=int
        # same duplicate question under the random stratified protocol, where id
        trr, var, ter = random_split(ctx["y"], SPLIT_FRACTIONS, seed)
        a2 = audit_split(ctx, trr, var, ter, "random_stratified_with_duplicates"
        store["audit"].append(a2)
        prep_r = prepare(ctx, trr, ter, seed)
        met, ft, secs = run_centralized_matched(prep_r, seed)
        log_run(store, "random_stratified_with_duplicates", "centralized_matched
        return store

    def unit_unseen(seed, family):
        store = new_store()
        k = FAM_TO_CODE[family]
        tr0, _, te0 = TEMPORAL_SPLIT
        tr = tr0[CTX["fam"][tr0] != k]                                      # f
        te = te0[(CTX["fam"][te0] == k) | (CTX["fam"][te0] == BENIGN_CODE)]   #
        check(not (CTX["fam"][tr] == k).any(), f"{family} absent from training."
        check(np.intersect1d(tr, te).size == 0, "Unseen-family train/test disjoi
        prep = prepare(CTX, tr, te, seed)
        cond = f"unseen_{family}"
        met, ft, secs = run_centralized_matched(prep, seed)
        log_run(store, cond, "centralized_matched", seed, prep, met, ft, len(pre
        fl_experiment(store, cond, prep, seed, ALPHA_FOR_ABLATIONS, False)
        return store

    def unit_split(seed, name, fr):
        store = new_store()
```

```
    tr, va, te = temporal_split(CTX["n"], fr)
    prep = prepare(CTX, tr, te, seed)
    met, ft, secs = run_centralized_matched(prep, seed)
    log_run(store, f"split_{name}", "centralized_matched", seed, prep, met,
    return store

def units_for(seed):
    u = [(f"s{seed}|core", functools.partial(unit_core, seed))]
    for a in ALPHAS:
        u.append((f"s{seed}|a{a:g}", functools.partial(unit_alpha, seed, a,
    if RUN_DUP_ABLATION:
        u.append((f"s{seed}|dups", functools.partial(unit_dups, seed)))
    for fam_ in UNSEEN_FAMILIES:
        u.append((f"s{seed}|unseen|{fam_}", functools.partial(unit_unseen, s
    if RUN_SPLIT_SENSITIVITY:
        for nm, fr in SPLIT_SENSITIVITY.items():
            u.append((f"s{seed}|split|{nm}", functools.partial(unit_split, s
    return u

def unit_path(key):
    return RUNS / (re.sub(r"[^A-Za-z0-9_.-]+", "__", f"{NOTEBOOK_VERSION}__{
print("Engine ready. Units per seed:", len(units_for(SEEDS[0])))
```

```
Engine ready. Units per seed: 10
```

```
# =========================
# 12. RUN EVERYTHING (resumable)
# =========================
STORE = {}
ALL_KEYS = []
T_START = time.time()
for seed in SEEDS:
    PREP.clear()
    for key, fn in units_for(seed):
        ALL_KEYS.append(key)
        path = unit_path(key)
        if path.exists():
            STORE[key] = json.load(open(path))
            print(f"[skip] {key} (already completed)")
            continue
        t = time.time()
        STORE[key] = fn()
        with open(path, "w") as fh:
            json.dump(STORE[key], fh, default=json_default)
        print(f"[done] {key}  {time.time() - t:6.1f}s   (total {(time.time()
    PREP.clear()
CTX_DUP.clear()
check(all(k in STORE for k in ALL_KEYS), f"All {len(ALL_KEYS)} experiment un
```

```
[done] s42|core   255.3s   (total 4.3 min)
[done] s42|a10    95.7s   (total 5.9 min)
[done] s42|a1    95.0s   (total 7.4 min)
[done] s42|a0.5    97.0s   (total 9.1 min)
[done] s42|a0.1    94.4s   (total 10.6 min)
[done] s42|dups   165.4s   (total 13.4 min)
```

```
[done] s42|unseen|DoS   128.3s   (total 15.5 min)
[done] s42|unseen|Reconnaissance   125.4s   (total 17.6 min)
[done] s42|split|50_25_25    46.4s   (total 18.4 min)
[done] s42|split|70_10_20    48.1s   (total 19.2 min)
[done] s123|core   235.6s   (total 23.1 min)
[done] s123|a10    79.7s   (total 24.4 min)
[done] s123|a1    80.1s   (total 25.8 min)
[done] s123|a0.5    81.6s   (total 27.1 min)
[done] s123|a0.1    80.7s   (total 28.5 min)
[done] s123|dups   140.2s   (total 30.8 min)
[done] s123|unseen|DoS   134.6s   (total 33.1 min)
[done] s123|unseen|Reconnaissance   128.9s   (total 35.2 min)
[done] s123|split|50_25_25    45.1s   (total 36.0 min)
[done] s123|split|70_10_20    46.0s   (total 36.7 min)
[done] s456|core   233.2s   (total 40.6 min)
[done] s456|a10    81.3s   (total 42.0 min)
[done] s456|a1    80.1s   (total 43.3 min)
[done] s456|a0.5    80.1s   (total 44.6 min)
[done] s456|a0.1    80.6s   (total 46.0 min)
[done] s456|dups   141.1s   (total 48.3 min)
[done] s456|unseen|DoS   125.8s   (total 50.4 min)
[done] s456|unseen|Reconnaissance   128.1s   (total 52.6 min)
[done] s456|split|50_25_25    45.3s   (total 53.3 min)
[done] s456|split|70_10_20    46.2s   (total 54.1 min)
[done] s789|core   248.9s   (total 58.2 min)
[done] s789|a10    80.6s   (total 59.6 min)
[done] s789|a1    82.3s   (total 61.0 min)
[done] s789|a0.5    81.1s   (total 62.3 min)
[done] s789|a0.1    82.3s   (total 63.7 min)
[done] s789|dups   138.4s   (total 66.0 min)
[done] s789|unseen|DoS   127.2s   (total 68.1 min)
[done] s789|unseen|Reconnaissance   128.1s   (total 70.2 min)
[done] s789|split|50_25_25    45.1s   (total 71.0 min)
[done] s789|split|70_10_20    46.1s   (total 71.8 min)
[done] s101112|core   250.3s   (total 75.9 min)
[done] s101112|a10    80.4s   (total 77.3 min)
[done] s101112|a1    81.5s   (total 78.6 min)
[done] s101112|a0.5    83.4s   (total 80.0 min)
[done] s101112|a0.1    85.4s   (total 81.4 min)
[done] s101112|dups   143.2s   (total 83.8 min)
[done] s101112|unseen|DoS   126.2s   (total 85.9 min)
[done] s101112|unseen|Reconnaissance   125.8s   (total 88.0 min)
[done] s101112|split|50_25_25    45.0s   (total 88.8 min)
[done] s101112|split|70_10_20    45.3s   (total 89.5 min)
[PASS] All 50 experiment units completed.
True
```

```
# =========================
# 13. AGGREGATION + STATISTICS
# =========================
METRICS = ["macro_f1", "precision", "recall", "specificity", "fpr", "pr_auc"
all_rows, all_family, all_curve, all_het, all_audit, all_meta = [], [], [],
for k in ALL_KEYS:
    s = STORE[k]
    all_rows += s["rows"]; all_family += s["family"]; all_curve += s["curve"
    all_het += s["hetero"]; all_audit += s["audit"]; all_meta += s["meta"]

rows_df = pd.DataFrame(all_rows)
```

```
fam_df = pd.DataFrame(all_family)
rows_df.to_csv(TABLES / "all_runs_raw.csv", index=False)
fam_df.to_csv(TABLES / "per_family_raw.csv", index=False)
pd.DataFrame(all_het).to_csv(TABLES / "client_heterogeneity.csv", index=Fals
pd.DataFrame(all_curve).to_csv(TABLES / "fedavg_curves_first_seed.csv", inde

# ---- seed-level values. Non-local methods have one run per (condition, see
# Local has one run per trained client, so it is collapsed TWO ways (both in
#   local_macro    : every trained client counts equally  (average monitorin
#   local_weighted : clients weighted by training rows     (traffic-level pe
# All local models are scored on the SAME global test set, so the weights ar
def _wmean(g, cols):
    w = g["train_rows"].to_numpy(float)
    out = {}
    for c in cols:
        v = g[c].to_numpy(float); ok = np.isfinite(v)
        out[c] = float(np.average(v[ok], weights=w[ok])) if ok.any() and w[c
    return pd.Series(out)

def seed_level_frame(d, cols, keys):
    parts = [d[d.method != "local"].groupby(keys + ["method", "seed"], dropn
    loc = d[d.method == "local"]
    if len(loc):
        mac = loc.groupby(keys + ["seed"])[cols].mean().reset_index(); mac["
        wt = loc.groupby(keys + ["seed"]).apply(lambda g: _wmean(g, cols)).r
        parts += [mac, wt]
    return pd.concat(parts, ignore_index=True)

seed_level = seed_level_frame(rows_df, METRICS, ["condition"])
n_local = (rows_df[rows_df.method == "local"].groupby(["condition", "seed"])
           .rename("local_clients_trained").reset_index())
seed_level.to_csv(TABLES / "seed_level_results.csv", index=False)
n_local.to_csv(TABLES / "local_clients_trained.csv", index=False)

def ci_half(sd, n):
    return float(stats.t.ppf(0.975, n - 1) * sd / np.sqrt(n)) if n > 1 and n

summ = []
for (cond, meth), g in seed_level.groupby(["condition", "method"]):
    for m in METRICS:
        v = g[m].dropna().to_numpy()
        n = len(v); sd = float(v.std(ddof=1)) if n > 1 else np.nan
        h = ci_half(sd, n)
        summ.append(dict(condition=cond, method=meth, metric=m, n_seeds=n, m
                         sd=sd, ci95_low=float(v.mean()) - h if n > 1 else n
                         ci95_high=float(v.mean()) + h if n > 1 else np.nan)
summary = pd.DataFrame(summ)
summary.to_csv(TABLES / "summary_by_condition.csv", index=False)

# ---- main table: Macro-F1, mean +/- SD over seeds
COND_ORDER = ["random_stratified", "temporal"] + [f"temporal_a{a:g}" for a i
              + ["temporal_with_duplicates", "random_stratified_with_duplicat
              + [f"unseen_{f}" for f in UNSEEN_FAMILIES] + [f"split_{n}" for
def cell(cond, meth, metric="macro_f1"):
    r = summary[(summary.condition == cond) & (summary.method == meth) & (su
```

```
    if r.empty or not np.isfinite(r["mean"].iloc[0]):
        return ""
    sd = r["sd"].iloc[0]
    return f"{r['mean'].iloc[0]:.4f} +/- {sd:.4f}" if np.isfinite(sd) else f
main_rows = []
for cond in COND_ORDER:
    matched = cell(cond, "centralized_matched")
    if cond.startswith("temporal_a") and not matched:
        matched = cell("temporal", "centralized_matched")      # same pooled
    main_rows.append({"condition": cond, "centralized_es(reference)": cell(c
                      "centralized_matched": matched, "local_macro": cell(co
                      "local_weighted": cell(cond, "local_weighted"), "fedav
main_table = pd.DataFrame(main_rows)
main_table.to_csv(TABLES / "main_table_macro_f1.csv", index=False)
print("MACRO-F1 (mean +/- SD over seeds)"); display(main_table)

# ---- paired comparisons: effect size + CI are primary; t-test and Wilcoxon
def sv(cond, meth, metric):
    r = seed_level[(seed_level.condition == cond) & (seed_level.method == me
    return r.set_index("seed")[metric].dropna()

def paired(label, A, B, metric):
    a, b = sv(*A, metric), sv(*B, metric)
    common = a.index.intersection(b.index)
    if len(common) < 2:
        return None
    d = (a.loc[common] - b.loc[common]).to_numpy()
    n = len(d); sd = d.std(ddof=1)
    t_p = stats.ttest_1samp(d, 0.0).pvalue if sd > 0 else np.nan
    try:
        w_p = float(stats.wilcoxon(d).pvalue) if np.any(d != 0) else np.nan
    except ValueError:
        w_p = np.nan
    dz = d.mean() / sd if sd > 0 else np.nan
    J = (1 - 3 / (4 * (n - 1) - 1)) if n >= 3 else np.nan
    h = ci_half(sd, n)
    return dict(comparison=label, A=f"{A[0]}/{A[1]}", B=f"{B[0]}/{B[1]}", me
                mean_diff=float(d.mean()), ci95_low=float(d.mean() - h), ci9
                hedges_g=float(dz * J) if np.isfinite(dz) and np.isfinite(J)
                cohen_dz=float(dz) if np.isfinite(dz) else np.nan,
                p_value=float(t_p) if np.isfinite(t_p) else np.nan,
                p_wilcoxon=w_p,
                wilcoxon_min_exact_p=float(2.0 ** (1 - n)))   # smallest two

def holm(pvals):
    p = np.asarray(pvals, float); out = np.full(len(p), np.nan)
    idx = [i for i in np.argsort(np.where(np.isnan(p), np.inf, p), kind="sta
    m_, running = len(idx), 0.0
    for rank, i in enumerate(idx):
        running = max(running, (m_ - rank) * p[i])
        out[i] = min(1.0, running)
    return out

specs = []
for a in ALPHAS:
```

```
    c = f"temporal_a{a:g}"
    specs += [(f"alpha={a:g}: FedAvg vs Local(macro-client)", (c, "fedavg"),
              (f"alpha={a:g}: FedAvg vs Local(sample-weighted)", (c, "fedavg
              (f"alpha={a:g}: FedAvg vs Centralized(matched)", (c, "fedavg")
              (f"alpha={a:g}: Local(macro-client) vs Centralized(matched)",
              (f"alpha={a:g}: Local(sample-weighted) vs Centralized(matched)
if len(ALPHAS) > 1:
    specs.append((f"FedAvg alpha={ALPHAS[0]:g} vs alpha={ALPHAS[-1]:g}",
                  (f"temporal_a{ALPHAS[0]:g}", "fedavg"), (f"temporal_a{ALPH
specs += [("Random stratified vs Temporal (centralized_es, reference)", ("ra
          ("Random stratified vs Temporal (centralized_matched)", ("random_s
if RUN_DUP_ABLATION:
    specs += [("Duplicates kept vs removed, random stratified split (central
              ("Duplicates kept vs removed, temporal (centralized_matched)",
              (f"Duplicates kept vs removed, temporal (FedAvg alpha={ALPHA_F
for f in UNSEEN_FAMILIES:
    c = f"unseen_{f}"
    specs += [(f"[aux] Unseen {f}: FedAvg vs Local(macro-client)", (c, "feda
              (f"[aux] Unseen {f}: FedAvg vs Local(sample-weighted)", (c, "f
              (f"[aux] Unseen {f}: FedAvg vs Centralized(matched)", (c, "fed
for n_ in SPLIT_SENSITIVITY:
    specs.append((f"Split {n_} vs 60/20/20 (centralized_matched)", (f"split_

pt = []
for metric in ["macro_f1", "recall", "specificity", "fpr", "pr_auc"]:
    block = [r for r in (paired(l, A, B, metric) for l, A, B in specs) if r]
    if block:
        ph = holm([b["p_value"] for b in block]); pw = holm([b["p_wilcoxon"]
        for b, x, y_ in zip(block, ph, pw):          # Holm step-down within
            b["p_holm"] = float(x) if np.isfinite(x) else np.nan
            b["p_wilcoxon_holm"] = float(y_) if np.isfinite(y_) else np.nan
    pt += block
paired_df = pd.DataFrame(pt)
paired_df.to_csv(TABLES / "paired_tests.csv", index=False)
PAIRED_COLS = ["comparison", "n_pairs", "mean_diff", "ci95_low", "ci95_high"
print("\nPAIRED COMPARISONS (Macro-F1; A - B; n = number of seeds). Primary:
if len(SEEDS) < 6:
    print(f"NOTE: with n={len(SEEDS)} pairs the smallest attainable two-side
          "Wilcoxon can never reach p < 0.05 below n = 6, so read it as a si
display(paired_df[paired_df.metric == "macro_f1"][PAIRED_COLS].round(4))

# ---- per-family DETECTION RECALL (binary detector: no per-family precision
fs = seed_level_frame(fam_df, ["recall", "specificity", "support"], ["condit
fam_sum = fs.groupby(["condition", "method", "family"])[["recall", "specific
fam_sum.columns = ["_".join(c) for c in fam_sum.columns]
fam_sum = fam_sum.reset_index()
fam_sum.to_csv(TABLES / "per_family_summary.csv", index=False)
_a_focus = min(ALPHAS, key=lambda a: abs(math.log(a / ALPHA_FOR_ABLATIONS)))
focus = f"temporal_a{_a_focus:g}"
print(f"\nPER-FAMILY DETECTION RECALL, condition={focus} (mean over seeds)")
piv = fam_sum[(fam_sum.condition == focus) & (fam_sum.family != "Benign")].p
sup = fam_sum[(fam_sum.condition == focus) & (fam_sum.family != "Benign") &
display(piv.assign(test_support=sup).round(4))

print("\nHETEROGENEITY ACHIEVED (mean over seeds)")
```

```
het_df = pd.DataFrame(all_het)
display(het_df.groupby("condition")[["js_divergence_mean", "attack_rate_sd",
                                    "clients_below_min_rows", "local_traine
```

```
MACRO-F1 (mean +/- SD over seeds)
```

	condition	centralized_es(reference)	centralized_mat
0	random_stratified	0.9399 +/- 0.0021	0.9358 +/- 0
1	temporal	0.9297 +/- 0.0011	0.9271 +/- 0
2	temporal_a10		0.9271 +/- 0
3	temporal_a1		0.9271 +/- 0
4	temporal_a0.5		0.9271 +/- 0
5	temporal_a0.1		0.9271 +/- 0
6	temporal_with_duplicates		0.9763 +/- 0
7	random_stratified_with_duplicates		0.9791 +/- 0
8	unseen_DoS		0.7138 +/- 0
9	unseen_Reconnaissance		0.7776 +/- 0
10	split_50_25_25		0.9134 +/- 0
11	split_70_10_20		0.9315 +/- 0

```
PAIRED COMPARISONS (Macro-F1; A - B; n = number of seeds). Primary: mean_di
NOTE: with n=5 pairs the smallest attainable two-sided exact Wilcoxon p is 0
```

	comparison	n_pairs	mean_diff	ci95_low	ci95_high	hedges_g	p
0	alpha=10: FedAvg vs Local(macro-client)	5	0.0052	0.0027	0.0076	2.0819	
1	alpha=10: FedAvg vs Local(sample-weighted)	5	0.0049	0.0022	0.0076	1.7792	
2	alpha=10: FedAvg vs Centralized(matched)	5	-0.0002	-0.0029	0.0025	-0.0663	
3	alpha=10: Local(macro-client) vs Centralized(m...	5	-0.0053	-0.0097	-0.0010	-1.2172	
4	alpha=10: Local(sample-weighted) vs	5	-0.0050	-0.0096	-0.0005	-1.1106	

	Centralize...					
5	alpha=1: FedAvg vs Local(macro-client)	5	0.0073	0.0002	0.0144	1.0159
6	alpha=1: FedAvg vs Local(sample-weighted)	5	0.0116	0.0067	0.0165	2.3456
7	alpha=1: FedAvg vs Centralized(matched)	5	-0.0067	-0.0137	0.0003	-0.9463
8	alpha=1: Local(macro-client) vs Centralized(ma...	5	-0.0139	-0.0185	-0.0093	-3.0179
9	alpha=1: Local(sample-weighted) vs Centralized...	5	-0.0182	-0.0256	-0.0108	-2.4544
10	alpha=0.5: FedAvg vs Local(macro-client)	5	-0.0024	-0.0236	0.0189	-0.1104
11	alpha=0.5: FedAvg vs Local(sample-weighted)	5	0.0100	-0.0078	0.0278	0.5600
12	alpha=0.5: FedAvg vs Centralized(matched)	5	-0.0258	-0.0502	-0.0015	-1.0530
13	alpha=0.5: Local(macro-client) vs Centralized(...	5	-0.0235	-0.0365	-0.0104	-1.7810
14	alpha=0.5: Local(sample-weighted) vs Centraliz...	5	-0.0358	-0.0694	-0.0023	-1.0605
15	alpha=0.1: FedAvg vs Local(macro-client)	5	0.0382	-0.0371	0.1136	0.5040
16	alpha=0.1: FedAvg vs Local(sample-weighted)	5	0.0253	-0.0067	0.0573	0.7865
17	alpha=0.1: FedAvg vs Centralized(matched)	5	-0.0307	-0.0543	-0.0071	-1.2938
18	alpha=0.1: Local(macro-client) vs Centralized(...	5	-0.0689	-0.1450	0.0071	-0.9003
19	alpha=0.1: Local(sample-weighted) vs Centraliz...	5	-0.0560	-0.0730	-0.0391	-3.2870
20	FedAvg alpha=10 vs alpha=0.1	5	0.0305	0.0062	0.0549	1.2445
21	Random stratified vs Temporal	5	0.0101	0.0084	0.0119	5.7559

Next steps: Generate code with `main_table` New interactive sheet

```
# =========================
# 14. FIGURES
# =========================
def stat(cond, meth, metric="macro_f1"):
    r = summary[(summary.condition == cond) & (summary.method == meth) & (su
    if r.empty: return np.nan, np.nan
    r = r.iloc[0]
    return r["mean"], (r["ci95_high"] - r["mean"]) if np.isfinite(r["ci95_hi

# Fig 1: Macro-F1 vs heterogeneity
fig, ax = plt.subplots(figsize=(8, 5))
xs = np.arange(len(ALPHAS))
for meth, lab, mk in [("fedavg", "FedAvg", "o"), ("local_macro", "Local (mac
    ys = [stat(f"temporal_a{a:g}", meth) for a in ALPHAS]
    ax.errorbar(xs, [y[0] for y in ys], yerr=[y[1] for y in ys], marker=mk,
cm, ch = stat("temporal", "centralized_matched")
if np.isfinite(cm):
    ax.axhline(cm, color="k", ls="--", label="Centralized (pooled, budget-ma
    ax.axhspan(cm - ch, cm + ch, color="k", alpha=0.08)
ax.set_xticks(xs); ax.set_xticklabels([f"alpha={a:g}" for a in ALPHAS])
ax.set_xlabel("client heterogeneity  (left = near-IID  ->  right = extreme n
ax.set_ylabel("Macro-F1 (mean, 95% CI over seeds)"); ax.set_title("Temporal
ax.grid(alpha=0.25); ax.legend(); plt.tight_layout(); plt.savefig(FIGURES /

# Fig 2: random vs temporal
fig, ax = plt.subplots(figsize=(7, 5)); w = 0.35
for j, meth in enumerate(["centralized_es", "centralized_matched"]):
    ys = [stat(c, meth) for c in ["random_stratified", "temporal"]]
    ax.bar(np.arange(2) + j * w, [y[0] for y in ys], w, yerr=[y[1] for y in
           label=meth + (" (reference)" if meth == "centralized_es" else "")
ax.set_xticks(np.arange(2) + w / 2); ax.set_xticklabels(["Random stratified"
ax.set_ylabel("Macro-F1"); ax.set_title("Random stratified vs temporal evalu
plt.tight_layout(); plt.savefig(FIGURES / "fig2_random_vs_temporal.png", dpi

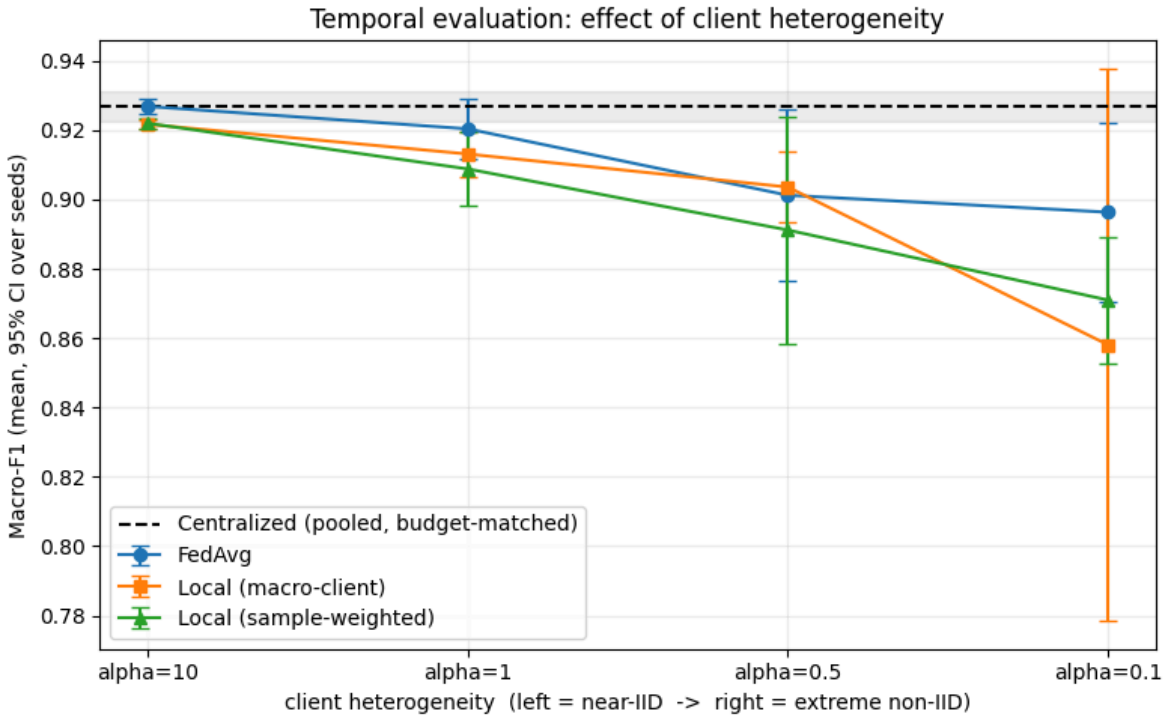
# Fig 3: per-family recall
sub = fam_sum[(fam_sum.condition == focus) & (fam_sum.family != "Benign")]
fams_ = sorted(sub.family.unique()); methods_ = [m for m in ["centralized_ma
fig, ax = plt.subplots(figsize=(11, 5)); w = 0.8 / max(len(methods_), 1)
for j, meth in enumerate(methods_):
    s_ = sub[sub.method == meth].set_index("family").reindex(fams_)
    ax.bar(np.arange(len(fams_)) + j * w, s_["recall_mean"], w, yerr=s_["rec
ax.set_xticks(np.arange(len(fams_)) + w * (len(methods_) - 1) / 2); ax.set_x
ax.set_ylabel("Detection recall (binary detector)"); ax.set_title(f"Per-fami
ax.legend(); ax.grid(alpha=0.25, axis="y"); plt.tight_layout(); plt.savefig(

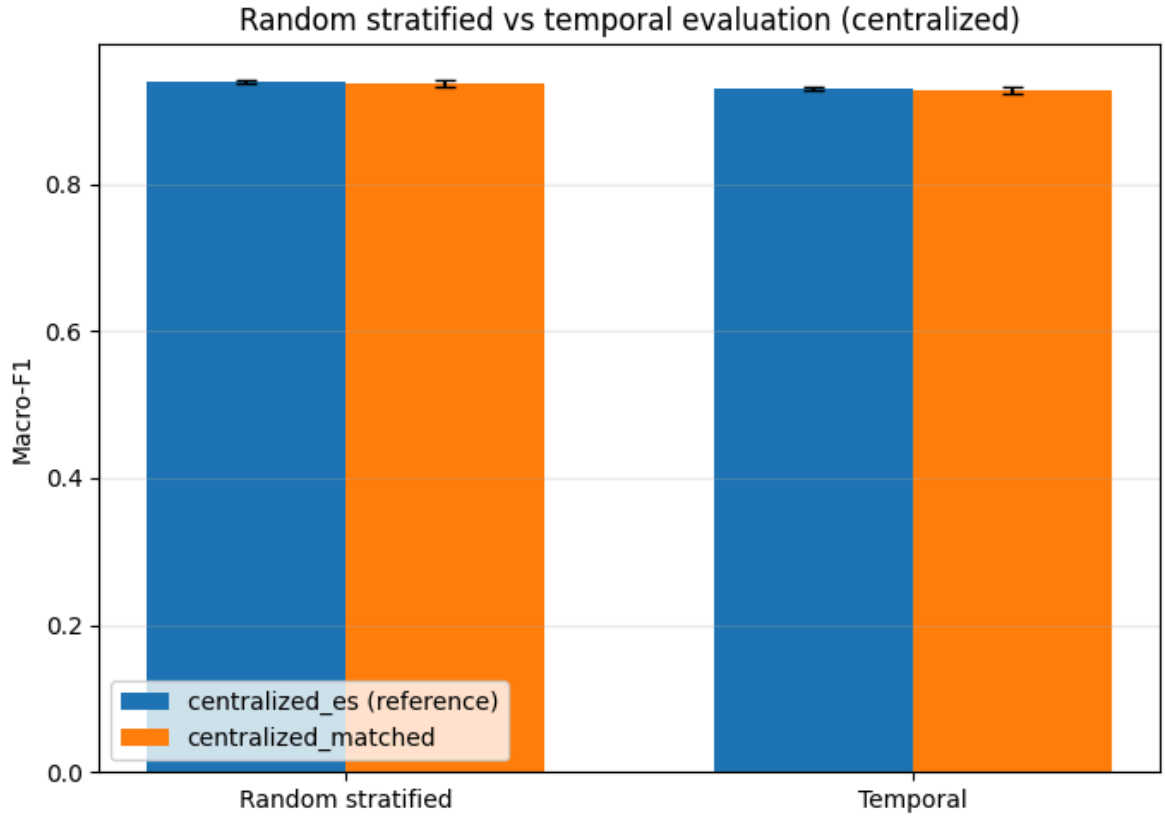
# Fig 4: duplicate ablation
if RUN_DUP_ABLATION:
    fig, ax = plt.subplots(figsize=(7, 5)); w = 0.35
    pairs = [("centralized_matched", "random_stratified", "random_stratified
             ("centralized_matched", "temporal", "temporal_with_duplicates")
             ("fedavg", f"temporal_a{_a_focus:g}", "temporal_with_duplicates
    for j, (cond, lab) in enumerate([(0, "duplicates removed"), (1, "duplica
        ys = [stat(p[1] if j == 0 else p[2], p[0]) for p in pairs]
        ax.bar(np.arange(3) + j * w, [y[0] for y in ys], w, yerr=[y[1] for y
```

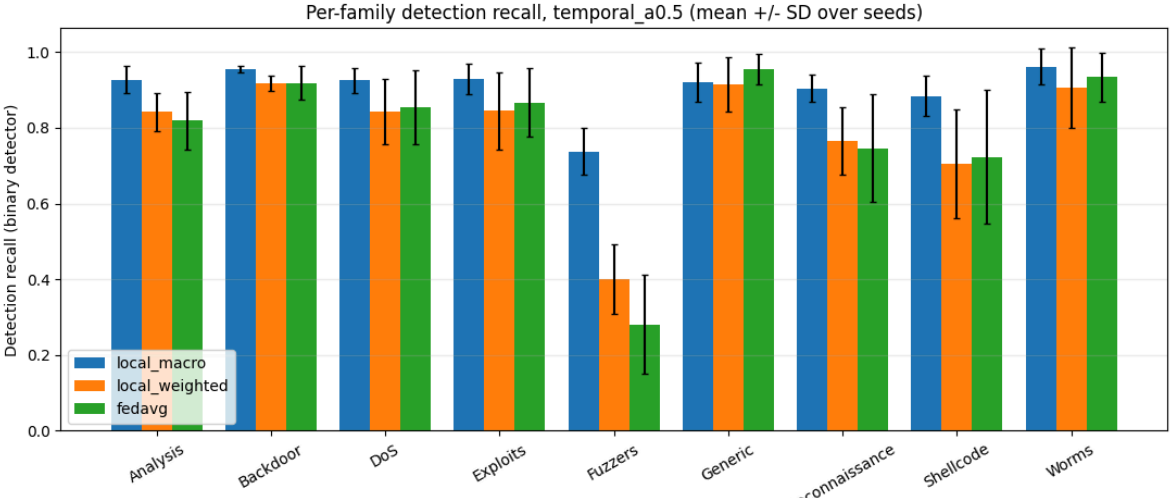
```
    ax.set_xticks(np.arange(3) + w / 2); ax.set_xticklabels(["Centralized, r
    ax.set_ylabel("Macro-F1"); ax.set_title("Duplicate sensitivity"); ax.leg
    plt.tight_layout(); plt.savefig(FIGURES / "fig4_duplicate_ablation.png",

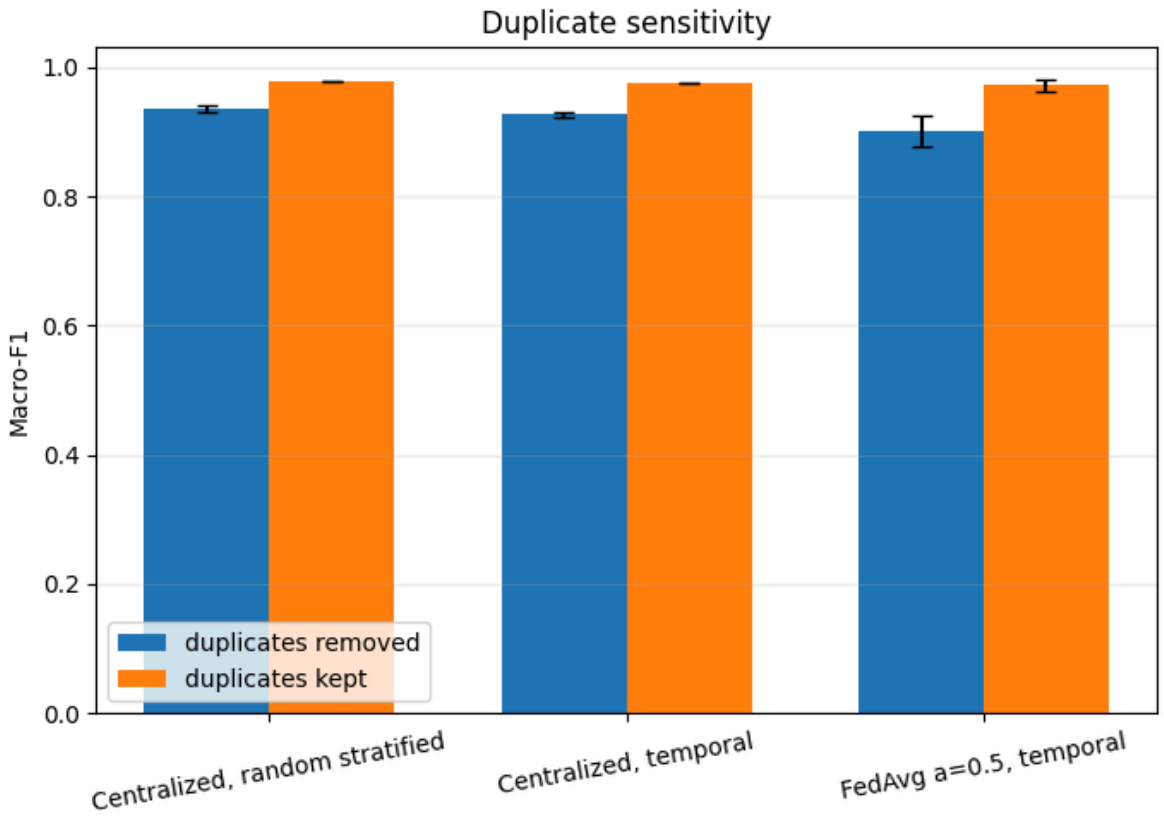
# Fig 5: FedAvg curves (first seed)
cur = pd.DataFrame(all_curve)
if len(cur):
    fig, ax = plt.subplots(figsize=(8, 5))
    for cond, g in cur.groupby("condition"):
        ax.plot(g["round"], g["macro_f1"], marker="o", label=cond)
    ax.set_xlabel("FedAvg round"); ax.set_ylabel("Macro-F1"); ax.set_title(f
    ax.grid(alpha=0.25); ax.legend(); plt.tight_layout(); plt.savefig(FIGURE

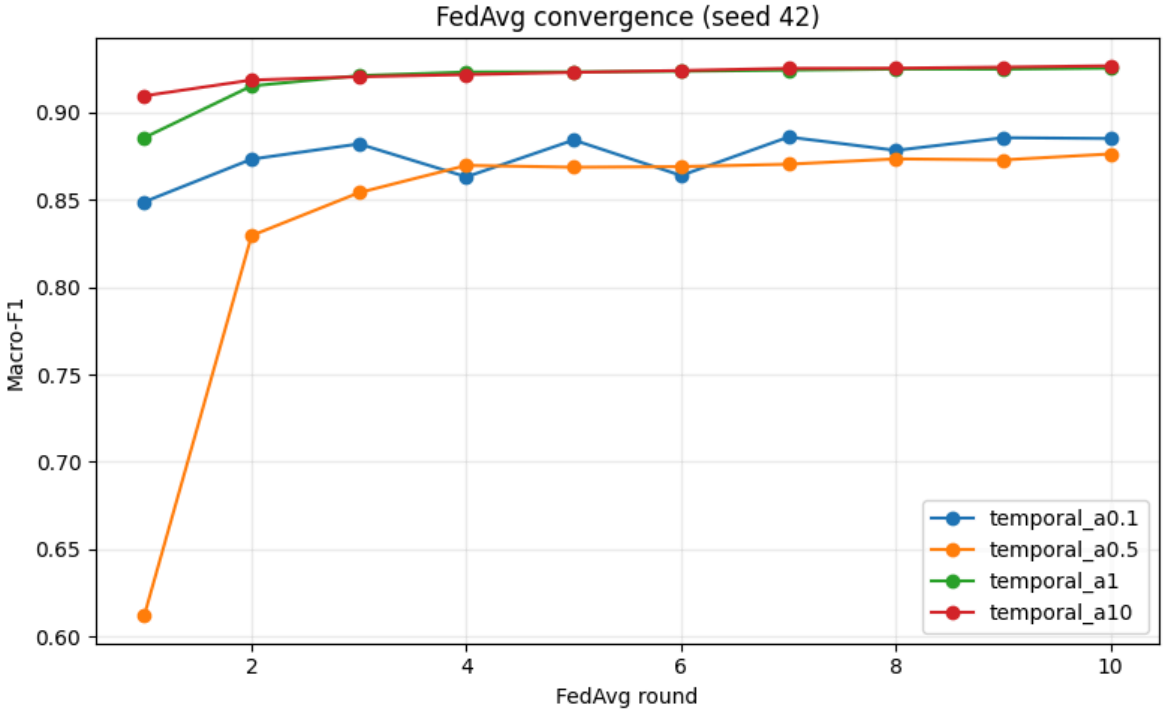
# Fig 6: unseen-family recall
if UNSEEN_FAMILIES:
    fig, ax = plt.subplots(figsize=(8, 5)); w = 0.2
    for j, meth in enumerate(["centralized_matched", "local_macro", "local_w
        vals, errs = [], []
        for f in UNSEEN_FAMILIES:
            r = fam_sum[(fam_sum.condition == f"unseen_{f}") & (fam_sum.meth
            vals.append(r["recall_mean"].iloc[0] if len(r) else np.nan); err
        ax.bar(np.arange(len(UNSEEN_FAMILIES)) + j * w, vals, w, yerr=np.nan
    ax.set_xticks(np.arange(len(UNSEEN_FAMILIES)) + 1.5 * w); ax.set_xtickla
    ax.set_ylabel("Recall on the held-out family"); ax.set_title("[Auxiliary
    ax.legend(); ax.grid(alpha=0.25, axis="y"); plt.tight_layout(); plt.save
```

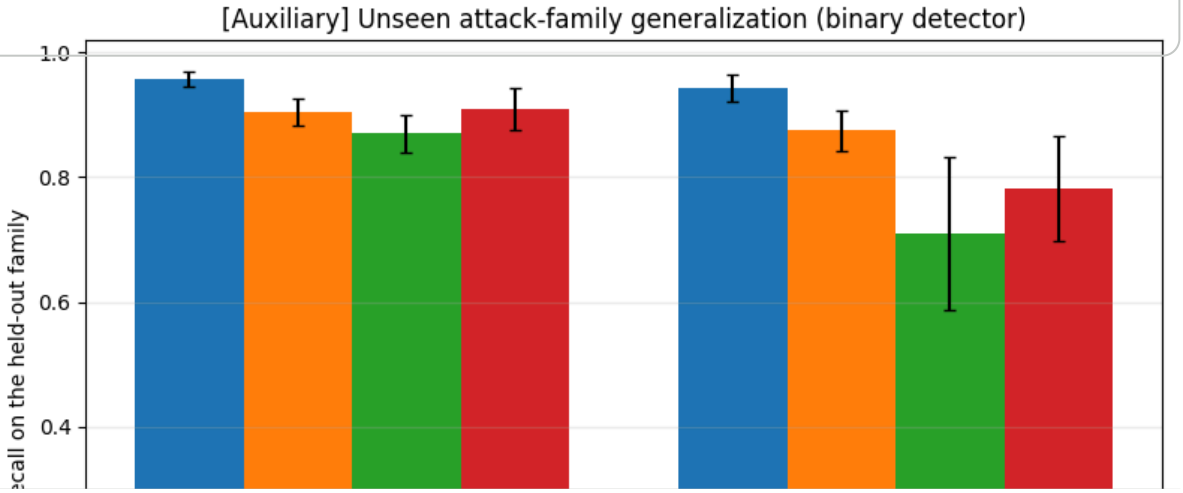

Temporal evaluation: effect of client heterogeneity
Macro-F1 (mean, 95% CI over seeds)
0.94
0.92
0.90
0.88
0.86
0.84
0.82
0.80
0.78
Centralized (pooled, budget-matched)
FedAvg
Local (macro-client)
Local (sample-weighted)
alpha=10
alpha=1
alpha=0.5
alpha=0.1
client heterogeneity (left = near-IID -> right = extreme non-IID)


Random stratified vs temporal evaluation (centralized)
Macro-F1
0.8
0.6
0.4
0.2
0.0
centralized_es (reference)
centralized_matched
Random stratified
Temporal


Per-family detection recall, temporal_a0.5 (mean +/- SD over seeds)
Detection recall (binary detector)
1.0
0.8
0.6
0.4
0.2
0.0
local_macro
local_weighted
fedavg
Analysis
Backdoor
DoS
Exploits
Fuzzers
Generic
Shellcode
Worms


Duplicate sensitivity
Macro-F1
1.0
0.8
0.6
0.4
0.2
0.0
duplicates removed
duplicates kept
Centralized, random stratified
Centralized, temporal
FedAvg a=0.5, temporal


FedAvg convergence (seed 42)
Macro-F1
0.90
0.85
0.80
0.75
0.70
0.65
0.60
2
4
6
8
10
FedAvg round
temporal_a0.1
temporal_a0.5
temporal_a1
temporal_a10


[Auxiliary] Unseen attack-family generalization (binary detector)
1.0
0.8
0.6
0.4

```
# ========================
# 15. FINAL GATES, LEAKAGE AUDIT, MANIFEST, REPORT
# ========================
audit_df = pd.DataFrame(all_audit + STATIC_AUDITS)
dedup_audits = audit_df[audit_df.context == "dedup"]
dup_audits = audit_df[audit_df.context == "with_duplicates"] if "context" i

hash_ok = bool((rows_df.groupby(["condition", "seed"]).test_hash.nunique()
gates = {
    "all_units_completed": bool(all(k in STORE for k in ALL_KEYS)),
    "seed_count_matches_profile": bool(rows_df.seed.nunique() == len(SEEDS)
    "metrics_finite": bool(np.isfinite(rows_df["macro_f1"]).all()),
    "identical_test_set_across_methods": hash_ok,
    "preprocessing_fit_on_train_only": bool(rows_df.leak_free.all()),
    "dedup_splits_have_no_train_test_duplicate_overlap": bool((dedup_audits
    "dedup_splits_have_no_index_overlap": bool((dedup_audits.index_overlap
    "fedavg_weights_finite": bool(all(m.get("fedavg_weights_finite", True)
    "static_leakage_checks": bool(all(LEAKAGE_STATIC.values())),
    "fedavg_present_for_every_alpha": bool(all(((rows_df.condition == f"ten
}
print("===== FINAL VERIFICATION =====")
for k, v in gates.items():
    print("[PASS]" if v else "[FAIL]", k)
check(all(gates.values()), "All final verification gates passed.")

leak_report = dict(
    static_checks=LEAKAGE_STATIC,
    preprocessing="one preprocessor per seed fitted on the capped TRAIN sam
    validation_split_use="reserved and untouched; no hyper-parameter or mod
    training_sample="seeded sample of the historical training block, strati
    split_overlap_audits=audit_df.to_dict("records"),
    interpretation=("train_test_dup counts test rows with an identical feat
                    "It must be 0 for the primary (deduplicated) dataset ar
                    "where it quantifies how identical records crossing the
                    "(duplicates are not claimed to be leakage per se; repe
)
json.dump(leak_report, open(RESULTS / "leakage_audit.json", "w"), indent=2,

for _fp in FILE_PROVENANCE:
    _fp["rows"] = row_counts.get(_fp["file"])

manifest = dict(
    created_utc=datetime.datetime.utcnow().isoformat() + "Z",
    notebook_version=NOTEBOOK_VERSION,
    run_profile=RUN_PROFILE,
    dataset=dict(name="UNSW-NB15", source="UNSW Canberra Cyber Range Lab (M
                 files=FILE_PROVENANCE, rows_loaded=ROWS_LOADED, invalid_ro
                 final_rows=ROWS_FINAL, sessions=int(len(sessions)), sessio
                 largest_observed_gap_hours=AUDIT["largest_gap_hours"],
                 duplicate_policy="remove (features+label+family identical;
                 identifier_features="excluded (srcip, dstip, sport, dsport
    split=dict(temporal=dict(train=SPLIT_FRACTIONS[0], validation=SPLIT_FRA
               random_stratified=dict(train=SPLIT_FRACTIONS[0], validation=
                          note="random stratified baseline; not claimed to
```

```
                sensitivity=SPLIT_SENSITIVITY),
    seeds=SEEDS, dirichlet_alphas=ALPHAS, clients=N_CLIENTS, partition_by=P
    model=dict(type="MLPClassifier", hidden_layers=list(HIDDEN), activation
                batch_size=256, learning_rate=1e-3, dtype="float32"),
    training=dict(max_train_rows=MAX_TRAIN_ROWS, fedavg_rounds=FED_ROUNDS,
                  centralized_matched_epochs=MATCHED_EPOCHS, centralized_es
                  local_training="fresh random init; MATCHED_EPOCHS epochs
                  local_aggregation="local_macro (equal client weight) and
                  training_sample_protocol="chronological holdout with seed
                  validation_use="reserved and untouched (no model selectio
                  partition_protocol="pure Dirichlet(alpha) label-skew over
                  partition_min_rows_constraint=PARTITION_MIN_ROWS, small_c
                  min_class_rows_local=MIN_CLASS_ROWS_LOCAL),
    preprocessing="Option A: federated training with shared feature preproc
    unseen_families=UNSEEN_FAMILIES, alpha_for_ablations=ALPHA_FOR_ABLATION
    tracked_curve_seeds=TRACK_CURVE_SEEDS,
    software=dict(python=sys.version.split()[0], sklearn=sklearn.__version_
                  numpy=np.__version__, scipy=scipy.__version__, platform=p
    verification_gates=gates,
    total_runtime_minutes=round((time.time() - T_START) / 60, 1),
)
json.dump(manifest, open(RESULTS / "experiment_manifest.json", "w"), indent

mf = paired_df[paired_df.metric == "macro_f1"].round(4)
wilcoxon_floor = 2.0 ** (1 - len(SEEDS))
report = (
    "UNSW-NB15 CENTRALIZED vs LOCAL vs FEDERATED IDS - EXPERIMENT REPORT\n"
    "=====================================================================\n
    f"Profile: {RUN_PROFILE} | seeds: {SEEDS} | alphas: {ALPHAS} | clients:
    f"Data lineage: {ROWS_LOADED:,} loaded -> {N_INVALID:,} invalid -> {N_I
    f"{len(sessions)} capture session(s); largest gap {AUDIT['largest_gap_h
    f"Unseen-family experiments: {UNSEEN_FAMILIES or 'none'}\n\n"
    "MACRO-F1 (mean +/- SD over seeds)\n" + main_table.to_string(index=Fals
    "PAIRED COMPARISONS, Macro-F1 (A - B; primary: mean difference, 95% CI,
    + mf[PAIRED_COLS].to_string(index=False) + "\n"
    f"(smallest attainable exact two-sided Wilcoxon p with n={len(SEEDS)} p
    "VERIFICATION GATES: " + ", ".join(f"{k}={v}" for k, v in gates.items()
    "LIMITATIONS THAT MUST APPEAR IN THE PAPER): "
```